

Báo cáo GPU FinOps & Cost Optimization

Sinh viên: Lương Anh Tuấn

MSSV: 2A202600113

Bài lab: Day 25 - GPU FinOps & Cost Optimization

Ngày nộp: 2026-05-13

1. Giới thiệu

Bài lab này mô phỏng một quy trình GPU FinOps thực tế, bao gồm giám sát tài nguyên GPU, theo dõi chi phí workload, sử dụng spot instance, áp dụng autoscaling, phát hiện chi phí lãng phí và so sánh hiệu quả huấn luyện trên GPU thật. Môi trường Docker Compose local cung cấp các service riêng cho quản lý cluster, billing, spot instance, autoscaler, cost tracker và API gateway. Notebook sau đó kết nối đến gateway từ Kaggle/Colab để chạy cả các kịch bản FinOps mô phỏng và workload GPU thật.

GPU FinOps rất quan trọng vì chi phí GPU có thể tăng nhanh, đặc biệt khi sử dụng các accelerator mạnh như A100 hoặc khi tài nguyên bị để idle. Mục tiêu chính là liên kết các chỉ số kỹ thuật như utilization, memory, runtime, power và scaling với các chỉ số tài chính như giá theo giờ, savings, waste và budget usage. Trong bài lab này, các chiến lược tối ưu nổi bật nhất là right-sizing, mixed precision training, dùng spot instance cho workload chịu được gián đoạn, autoscaling và báo cáo waste định kỳ.

2. Phân tích từng phần

Part 1: GPU Cluster Monitoring

Mock cluster ban đầu có 4 node và 8 GPU, gồm các loại T4, A100 và V100. Ở thời điểm giám sát ban đầu, toàn bộ 8 GPU đều idle, average utilization chỉ đạt 9.0%, memory usage là 9.4 GB trên tổng 288.0 GB, và total power draw là 268 W. Đây là một dấu hiệu FinOps cần chú ý: tài nguyên vẫn đang tiêu thụ điện và chi phí, nhưng giá trị khai thác còn thấp do utilization quá nhỏ.

Kết quả monitoring cho phép quan sát từng GPU riêng lẻ, bao gồm utilization, memory, power và temperature. Điều này hữu ích vì metric tổng hợp ở cấp cluster có thể che giấu tình trạng mất cân bằng, ví dụ một GPU type bị dùng nhiều trong khi các GPU khác vẫn idle.

Bằng chứng:

- screenshots/part1_cluster_monitoring.png
- screenshots/part1_cluster_metrics_summary.png

Part 2: Workload Submission & Cost Tracking

Bốn workload đã được submit gồm ResNet training, BERT training, inference API và một workload LLM training lớn hơn. Sau khi submit, 5 trên 8 GPU chuyển sang trạng thái busy và average utilization tăng lên 51.7%. Kết quả này cho thấy cluster đủ capacity cho nhóm workload ban đầu, nhưng vẫn còn 3 GPU idle.

Billing được ghi nhận cho cả workload on-demand và spot. Total cost là \$1.1949, total savings là \$1.2927 và budget used chỉ ở mức 1.2%. Khoản tiết kiệm lớn nhất đến từ workload LLM chạy bằng spot, tiết kiệm \$1.2845 so với on-demand equivalent. Điều này cho thấy chính sách placement workload có ảnh hưởng rất lớn đến chi phí, ngay cả khi workload kỹ thuật không thay đổi.

Bằng chứng:

- screenshots/part2_workload_submission.png
- screenshots/part2_billing_summary.png

Part 3: Spot Instance Management

Spot pricing cho thấy cả ba loại GPU đều có mức discount so với on-demand:

GPU Type	On-Demand	Spot Price	Discount	Availability
T4	\$0.35/hr	\$0.2649/hr	24.3%	High
A100	\$3.67/hr	\$2.1949/hr	40.2%	Medium
V100	\$2.48/hr	\$1.6625/hr	33.0%	High

Ba spot request đã được cấp phát thành công: hai T4 instance và một A100 instance. Trong sự kiện mô phỏng preemption, không có instance nào bị thu hồi, nên cả 3 instance vẫn active. Báo cáo spot cho thấy spot cost là \$0.0003, on-demand equivalent là \$0.0009, tổng tiết kiệm \$0.0006 tương đương 62.2%.

Bài học chính là spot instance rất hiệu quả cho các workload fault-tolerant hoặc có checkpoint, nhưng không nên áp dụng một cách máy móc cho mọi job. Lợi ích chi phí cao, nhưng workload production cần tính đến rủi ro bị preempt.

Bằng chứng:

- screenshots/part3_spot_pricing.png
- screenshots/part3_spot_request.png
- screenshots/part3_spot_preemption.png

Part 4: Autoscaling Behavior

Autoscaling policy ban đầu có scale-up threshold 80%, scale-down threshold 20%, cooldown 60 giây, max 8 node, min 1 node, preferred GPU type là T4 và cost-aware scaling được bật. Sau khi cập nhật, autoscaler đánh giá với scale-up threshold 70% và scale-down threshold 25%.

Tại mức utilization 51.7%, autoscaler chọn **NO_ACTION** vì utilization nằm giữa ngưỡng scale-down và scale-up. Trong 5 evaluation cycle, quyết định giữ nguyên và node count vẫn là 4. Đây là hành vi phù hợp vì cluster đang được sử dụng ở mức trung bình, chưa cần tăng thêm capacity và cũng chưa nên scale down quá sớm.

Bằng chứng:

- screenshots/part4_autoscaler_policy.png
- screenshots/part4_autoscaler_evaluation.png

Part 5: Cost Analysis & Optimization

Cost tracker ghi nhận 5 snapshot. Mỗi snapshot có total interval cost là \$0.038056, idle cost là \$0.008833 và waste là 23.2%. Waste report tổng hợp total idle cost \$0.044165 trên total cost \$0.190280, potential monthly save là \$2,289.51 và severity ở mức **LOW**.

Các recommendation chính:

Priority	Recommendation	Expected Saving
Medium	Dùng spot instance cho workload fault-tolerant	65.0%

Priority	Recommendation	Expected Saving
Low	Chạy training job không khẩn cấp vào off-peak hours	20.0%

Dashboard tổng hợp hiển thị 8 GPU trên 4 node, utilization 51.7%, 5 GPU busy, 3 GPU idle, spend \$1.1949, savings \$1.2927 và waste 23.2%. Đây là góc nhìn vận hành quan trọng vì nó kết hợp tình trạng kỹ thuật với tình trạng chi phí.

Bằng chứng:

- screenshots/part5_cost_snapshots.png
- screenshots/part5_waste_report.png
- screenshots/part5_recommendations.png
- screenshots/part5_dashboard.png

Part 6: Visualization

Notebook đã tạo các biểu đồ cost breakdown và time-series cost tracking. Biểu đồ cost breakdown giúp so sánh phân bổ chi phí theo workload và GPU category. Biểu đồ time-series cho thấy total cost, idle cost và waste percentage thay đổi qua các snapshot.

Visualization rất hữu ích trong FinOps vì xu hướng thường quan trọng hơn một giá trị đơn lẻ. Một waste percentage tại một thời điểm có thể chỉ là tạm thời, nhưng nếu idle cost lặp lại theo thời gian thì đó là tín hiệu cần tối ưu scheduling, right-sizing hoặc autoscaling.

Biểu đồ đã tạo:

- generated_charts/finops_cost_breakdown.png
- generated_charts/finops_timeseries.png

Bằng chứng:

- screenshots/part6_cost_breakdown_viz.png
- screenshots/part6_timeseries_viz.png

Part 7: Complete FinOps Workflow

Full workflow kết nối các bước trước đó thành một vòng lặp tối ưu hoàn chỉnh:

1. Trạng thái ban đầu: 8 GPU, utilization 51.7%, 3 GPU idle.
2. Submit workload nặng làm utilization tăng lên 74.7%, 8/8 GPU busy.
3. Autoscaler quyết định scale_up vì utilization vượt ngưỡng 70.0%.
4. Cost interval là \$0.040000 với waste 4.9%.
5. Recommendation gồm dùng spot instance và scheduling workload vào off-peak hours.
6. Spot optimization tạo ra \$0.0371 savings, tương đương 70.0%.
7. Final billing đạt total spend \$1.3640 và total saved \$1.4151, budget used 1.4%.

Workflow này thể hiện rõ chu trình FinOps: đo lường, phát hiện waste, đề xuất tối ưu, áp dụng tối ưu và xác minh kết quả. Nó cũng cho thấy FinOps nên là hoạt động liên tục, không phải chỉ phân tích một lần.

Bằng chứng:

- screenshots/part7_full_workflow.png

Part 8: Real GPU Workload trên Kaggle/Colab

Môi trường GPU thật phát hiện Tesla T4 với 15.6 GB memory, CUDA 12.8 và giả định pricing \$0.35/hr. Diagnostic test xác nhận pynvml khả dụng, memory ban đầu 472 MB, idle power 10.5 W và temperature 40 C.

Thí nghiệm training dùng ResNet-18 trên CIFAR-10 và so sánh FP32 với mixed precision AMP:

Metric	FP32	AMP	Kết quả
Total time	122.6 s	59.4 s	Nhanh hơn 2.06x
Peak memory	0.82 GB	0.60 GB	Giảm 0.22 GB
Estimated cost	\$0.011917	\$0.005774	Tiết kiệm \$0.006144
Cost saving	-	-	51.6%
Average GPU utilization	95.5%	91.4%	Cả hai đều tận dụng GPU tốt
Average power	66.7 W	65.1 W	AMP thấp hơn nhẹ

Nếu extrapolate ở quy mô lớn hơn, savings trở nên đáng kể: tiết kiệm \$4.33 cho 1 ngày training, \$30.31 cho 1 tuần và \$129.92 cho 1 tháng. AMP giữ độ chính xác gần tương đương trong khi giảm runtime, memory và cost, vì vậy đây là tối ưu low-risk hiệu quả nhất trong phần GPU thật.

Chi phí GPU thật cũng được report lại vào FinOps gateway. Project real GPU ghi nhận 2 workload, total cost \$0.013600 và savings \$0.004000. Dashboard cuối cùng vẫn ở trạng thái an toàn: total platform cost \$1.3640, total savings \$1.4151, budget utilization 1.4% và alert status OK.

Biểu đồ đã tạo:

- generated_charts/real_gpu_comparison.png
- generated_charts/real_gpu_telemetry.png
- generated_charts/cost_per_epoch.png

Bảng chứng:

- screenshots/part8_gpu_detection.png
- screenshots/part8_gpu_metrics_diagnostic.png
- screenshots/part8_fp32_summary.png
- screenshots/part8_amp_summary.png
- screenshots/part8_fp32_vs_amp_comparison.png
- screenshots/part8_real_gpu_cost_report.png

Part 8.5: Advanced GPU Cost Optimization

Phần nâng cao phân tích multi-GPU scaling, project forecasting, prioritization strategy và challenge optimization cuối cùng.

Với multi-GPU scaling, cấu hình có cost-performance tốt nhất là 8x A100, hoàn thành baseline single-GPU 2 giờ trong 0.36 giờ, chi phí \$10.49 và scaling efficiency 70.0%. Kết quả này cho thấy tăng số GPU có thể giảm wall-clock time, nhưng efficiency giảm dần do overhead communication và parallelization. Lựa chọn đúng phụ thuộc vào mục tiêu: hoàn thành nhanh nhất, chi phí thấp nhất hoặc cân bằng giữa deadline và budget.

Project forecast ước tính:

Forecast Metric	Value
Base expected cost	\$3,551.20

Forecast Metric	Value
Contingency, 20%	\$710.24
Forecast with buffer	\$4,261.44
95% confidence interval	\$3,573.51 - \$4,949.37
Best / worst case	\$3,289.06 - \$5,233.82

Optimization opportunity analysis bắt đầu từ baseline cost \$1,468.00 và ước tính nếu áp dụng toàn bộ strategy thì final cost còn \$179.68. Tổng savings là \$1,288.32, tương đương 87.8%. Phần này cho thấy savings nên được tính theo dạng multiplicative, không cộng tuyến tính, vì mỗi optimization làm thay đổi cost base còn lại.

Challenge scenario là fine-tuning LLM với baseline 8x A100 trong 200 giờ, baseline cost \$5,872.00, budget \$5,000.00 và deadline 2 tuần. Strategy được chọn là resize từ 8x xuống 4x A100, sau đó áp dụng AMP, batch size optimization và early stopping. Spot instance bị loại vì rủi ro preemption cao, không phù hợp với constraint chỉ chấp nhận risk mức medium. Forecast cuối là \$3,235.34 với 95% confidence interval từ \$2,709.42 đến \$3,761.27, vượt qua cả budget chính và upper-bound budget check. Optimized training time là 178.5 giờ, và 20% pessimistic duration là 214.2 giờ so với deadline 336 giờ.

Biểu đồ đã tạo:

- generated_charts/multi_gpu_scaling.png
- generated_charts/project_forecast.png
- generated_charts/optimization_roadmap.png
- generated_charts/advanced_finops_dashboard.png

Bảng chứng:

- screenshots/part85_multi_gpu_analysis.png
- screenshots/part85_project_forecast.png
- screenshots/part85_optimization_analysis_part_1.png
- screenshots/part85_optimization_analysis_part_2.png
- screenshots/part85_integrated_dashboard.png
- screenshots/part85_challenge_strategy.png

3. Kết luận và bài học rút ra

Bài lab cho thấy GPU FinOps cần kết hợp cả đo lường kỹ thuật và phân tích tài chính. Các tín hiệu kỹ thuật quan trọng gồm utilization, memory usage, runtime, power draw, autoscaling decision và preemption risk. Các tín hiệu tài chính quan trọng gồm hourly rate, spot discount, idle cost, total spend, savings, budget usage và forecast uncertainty.

Các chiến lược tối ưu hiệu quả nhất:

Strategy	Trường hợp phù hợp	Trade-off chính
Mixed precision AMP	Training deep learning trên GPU hỗ trợ AMP	Cần kiểm tra accuracy và numerical stability
Spot instance	Job fault-tolerant, có checkpoint, retry được	Có rủi ro preemption
Autoscaling	Nhu cầu thay đổi, queue workload tăng giảm	Cần threshold và cooldown hợp lý

Strategy	Trường hợp phù hợp	Trade-off chính
Right-sizing GPU count	Training có deadline cụ thể	Nhiều GPU hơn có thể nhanh hơn nhưng efficiency giảm
Waste reporting	Phát hiện idle capacity và kiểm soát budget	Cần monitoring liên tục
Forecasting with confidence intervals	Lập kế hoạch project và duyệt budget	Phụ thuộc vào estimate uncertainty

Kết luận quan trọng nhất là nên bắt đầu tối ưu bằng các thay đổi có rủi ro thấp nhưng tác động cao. AMP đã giảm chi phí training thật 51.6% và đồng thời rút ngắn runtime. Spot instance tạo savings lớn trong mock platform, nhưng chỉ phù hợp khi workload chịu được gián đoạn. Autoscaling và waste report là các guardrail cần thiết vì một GPU cluster có thể vẫn hoạt động bình thường về mặt kỹ thuật nhưng gây lãng phí nếu idle capacity không được nhìn thấy.

Trong project thực tế, workflow này có thể áp dụng trước và trong quá trình training GPU: ước tính chi phí, chọn GPU type và GPU count, chạy benchmark nhỏ, so sánh precision mode, đặt budget alert, checkpoint workload nếu dùng spot và theo dõi utilization xuyên suốt quá trình chạy. Quy trình này giúp team hoàn thành training trong budget mà vẫn giữ được độ tin cậy và chất lượng model.